**EXPERIMENT-13**

**Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case**

## Aim:

To study the limitations of low-code development platforms in implementing complex enterprise applications by analyzing their scalability, customization, security, integration, performance, and maintenance challenges using a real-world insurance claim management system as a case study.

## Overview:

Low-code development platforms allow users to build applications with minimal hand-written code through visual development tools, drag-and-drop components, predefined templates, workflows, and automation features.

Zoho Creator is an example of a low-code platform that supports form creation, report generation, workflow automation, data management, and application deployment. It helps organizations develop business applications quickly while reducing development effort.

However, complex enterprise applications often require advanced business logic, large-scale data processing, high availability, detailed access control, integration with multiple external systems, and extensive customization. Low-code platforms may introduce limitations when these requirements exceed their built-in capabilities or platform constraints.

This experiment examines these limitations through a case study of an Insurance Claim Management System developed using Zoho Creator. The study identifies the challenges faced when expanding a simple application into an enterprise-level solution and discusses possible approaches to overcome them.

## Case Study: Insurance Claim Management System Using Zoho Creator:

### 1. Introduction to the Case Study:

An insurance company needs an application to manage customer information, insurance policies, claim submissions, claim verification, approval workflows, and notifications.

The company initially chooses Zoho Creator because it provides a visual application builder, database forms, reports, workflow automation, and role-based access controls.

The initial application contains three major forms:

- **Customer Details:** Stores customer names, contact information, and other relevant details.
- **Insurance Policy:** Stores policy numbers, coverage amounts, policy types, and validity dates.
- **Insurance Claim:** Stores claim amounts, policy references, claim status, and approved amounts.

The application also uses workflows to validate claims, check remaining policy coverage, send email notifications, generate reports, and control access through user roles.

As the company grows, it wants to extend the application to support thousands of customers, large numbers of claims, integration with external systems, advanced fraud detection, and detailed auditing. These requirements reveal several limitations that must be considered.

## 2. Limitations of Low-Code Platforms

### A. Scalability and Performance

As the number of customers, policies, and claims increases, the application must process more records and execute more workflows.

For example, a claim validation workflow may retrieve policy information and calculate previously approved claims. If the workflow repeatedly queries large datasets, response times may increase.

**Limitation:** Performance depends on data volume, query design, platform limits, and workflow complexity. Built-in tools may not provide the same degree of low-level performance control as a fully customized application.

**Possible solution:** Optimize queries, avoid unnecessary repeated operations, use appropriate data models, and move computationally intensive processing to a dedicated backend when necessary.

### B. Limited Customization

Low-code platforms provide predefined components for forms, reports, dashboards, and workflows. These components simplify development but may restrict specialized functionality.

For example, an insurance company may require a highly customized claim investigation interface with complex visualizations, document comparisons, and interactive fraud-analysis tools.

**Limitation:** Such interfaces may require custom scripting or external frontend development beyond the platform's standard capabilities.

**Possible solution:** Use supported custom pages, scripts, APIs, or a separately developed frontend where appropriate.

**C. Complex Business Logic**

Insurance processing involves multiple rules, including policy eligibility, claim limits, duplicate detection, approval hierarchies, and exceptions.

For example, a claim may require additional verification when the requested amount exceeds a threshold or when multiple claims are submitted for the same policy within a short period.

**Limitation:** As the number of conditions and workflows increases, the application becomes harder to understand, debug, test, and maintain.

**Possible solution:** Divide business logic into reusable functions, document workflows, implement clear validation rules, and use automated tests where supported.

**D. Integration with External Systems**

Enterprise insurance applications may need to connect with customer relationship management systems, payment gateways, accounting software, document storage, identity verification services, and external insurance databases.

**Limitation:** Integrations depend on available APIs, authentication methods, rate limits, data formats, and connector support. Complex integrations may require additional development and monitoring.

**Possible solution:** Use supported APIs, secure connections, integration services, and a dedicated middleware layer when multiple systems must communicate reliably.

**E. Security and Access Control**

Insurance applications store sensitive customer information, policy documents, and claim details. Different users require different levels of access.

For example, an insurance agent may need to view assigned claims, while an administrator may require broader access to manage users and application settings.

**Limitation:** Security configuration can become difficult when access depends on multiple roles, record ownership rules, organizational hierarchies, and external identity systems. Incorrect configuration may expose information to unauthorized users.

**Possible solution:** Apply least-privilege access, review permission sets and sharing rules, test access with different user accounts, and use supported authentication and auditing mechanisms.

**3. Proposed Improvements**

The following approaches can help address the limitations identified in the case study:

1. **Optimize data retrieval:** Use efficient criteria-based queries and avoid unnecessary database operations.
2. **Improve application design:** Organize forms, relationships, and workflows logically.
3. **Use modular business logic:** Separate complex claim validation and approval processes into reusable components.
4. **Strengthen security:** Configure roles, permission sets, sharing rules, and appropriate access restrictions.
5. **Integrate external services carefully:** Use secure APIs and monitor failed requests and integration limits.
6. **Adopt staged deployment:** Test application changes in a staging environment before publishing them to production.
7. **Monitor performance:** Evaluate execution times, record volumes, and platform usage against actual requirements.
8. **Use a hybrid architecture when needed:** Retain Zoho Creator for forms and business workflows while moving specialized processing to a dedicated backend service.

**4. Discussion**

The case study demonstrates that Zoho Creator is suitable for rapidly developing and deploying standard business applications such as customer management, insurance policy tracking, claim submission, and notification workflows.

However, as the application evolves into a large enterprise system, the development team must evaluate platform limits involving scalability, customization, integration, security, cost, and maintainability.

A low-code platform should therefore be selected based on the application's current and future requirements rather than development speed alone. For complex systems, a hybrid architecture combining low-code tools with conventional software components may provide a practical balance between productivity and flexibility.

## Result:

Thus, the limitations of low-code development platforms for complex enterprise applications were studied through the Insurance Claim Management System case study. The major challenges identified included scalability, performance, customization, complex business logic, external integration, security, vendor dependency, maintenance, and cost. Suitable optimization strategies and hybrid development approaches were identified to address these challenges.